Vision-Based ADAS Dashboard

Classical Computer Vision vs. Hybrid Deep Learning

Computer Vision course project – M.Sc. Control Systems Engineering, University of Padova
Author: Kiavash Rabbani Soudabi (ID 2141822) – Supervisor: Prof. Pietro Zanuttigh – A.Y. 2025–2026

Purpose

Build a working driver-assistance (ADAS) pipeline that processes real dash-cam footage and renders an operator dashboard on top of it, covering four tasks: **lane detection** (drivable road boundaries), **lane classification** (solid vs. dashed), **Forward Collision Warning** (vehicle proximity) and **Lane Departure Warning** (lateral drift of the host vehicle).

The design question behind the project: learned end-to-end models dominate lane extraction in production ADAS, but relying on them as a black box skips the fundamentals. The solution chosen is a **hybrid architecture** – YOLO performs object detection, where deep learning genuinely excels, while the whole lane-extraction stack (colour filtering, edge gradients, geometric clustering, temporal tracking) is implemented from classical computer-vision primitives.

Pipeline

- **Illumination normalisation:** CLAHE on the L channel in LAB space corrects harsh shadows before thresholding.
- **Feature masking:** adaptive HLS colour masks (white and yellow) fused with directional edge gradients; Sobel was preferred over Canny because lane edges have a known orientation.
- **ROI and vehicle blackout:** a parameterised triangular region of interest, with detected vehicles removed so their edges cannot contaminate the lane fit.
- **Geometric lane extraction:** Hough transform, each line reduced to a compact 2-D feature vector.
- **Classification and tracking:** DBSCAN clustering over the feature space plus a *Paint-Fill Ratio* (above 85 % $\Rightarrow$ solid) to classify the marking, with temporal memory to suppress flicker.
- **ADAS HUD:** drivable-area polygon with asphalt validation, vehicle lane assignment (main lane / adjacent lane / out of bounds), forward-collision and lane-departure alerts.

Tools

Python, OpenCV, NumPy, scikit-learn (DBSCAN, StandardScaler), Ultralytics YOLOv8n for object detection, Matplotlib, Jupyter, LaTeX for the report.

Results and limitations

The pipeline reaches an estimated $\approx$ **75 % reliability** under reasonable road conditions: the lane overlay and the alerts behave correctly on the majority of frames, and solid/dashed classification and vehicle lane assignment are stable across the test scenes. Two limitations are documented explicitly: **speed** – YOLOv8n inference plus the classical chain in Python is a real bottleneck for deployment – and **curvature**: the linear lane model ($y = mx + b$) degrades on sharp bends, where a polynomial fit is needed. Working through feature extraction, clustering and temporal tracking from scratch made concrete what learned representations replace.

Repository contents

`notebook/FinalProject.ipynb` (executed project notebook) · `src/project.py` (documented pipeline module) · `report/` (LaTeX source, figures, compiled report PDF) · `data/roads/` (test images and 720p test video) · `media/` (720p dashboard demo) · `results/` and `docs/` (result figures and this overview).

Skills demonstrated

Computer vision fundamentals (colour spaces, CLAHE, thresholding, Sobel/Hough geometry); clustering-based model fitting (DBSCAN); temporal filtering and object tracking; deep-learning object detection (YOLOv8) integrated with a classical pipeline; ADAS reasoning (drivable area, forward-collision and lane-departure warnings); scientific reporting in LaTeX.